

Integrity



What will you learn

- Type Constraints
- Attribute Constraint
- Relvar Constraint
- Database Constraint
- The Golden Rule
- State vs. Transition Constraints
- Keys
- Referential Integrity
- Assertions
- Triggers

1. Introduction

- **Integrity** refers to the **accuracy or correctness** of data in the database.
- the DBMS needs to be informed of such **constraints**, and of course needs to enforce them somehow.

e.g. Status values must be in the range 1 to 100

```
CONSTRAINT SC3 IS_EMPTY ( STATUS<1 OR  
STATUS>100);
```

- Basically by rejecting any update that would otherwise violate them.

Introduction...

- When a new constraint is declared, The system must first make sure the database currently satisfies it;
If not, the new constraint is rejected;
Otherwise it is accepted and enforced from that point forward.
- DROP CONSTRAINT SC3;
- We are concerned very specifically with declarative integrity support
- declarative integrity support is important
 - Relieve the burden
 - More productive

Introduction...

A Constraint Classification Scheme:

- **Type** constraint
Specifies the legal value for a given type.
- **Attribute** constraint
Specifies the legal values for a given attribute.
- **Relvar** constraint
Specifies the legal values for a given relvar.
- **Database** constraint
Specifies the legal values for a given database.

2.Type Constraints

- Type constraints are the most elementary form of integrity constraint. just a enumeration of the legal values of the type

e.g.

```
CREATE DOMAIN COLOR# CHAR(6) DEFAULT '???'  
CONSTRAINT VALID_COLORS  
CHECK (VALUE IN ('RED', 'YELLOW', 'GREEN', '???'));  
CREATE TABLE P(...,COLOR COLOR#,...);
```

- A type constraints is a specification of the values that make up the type in question.
- Type constraints are **checked immediately**.
- No relvar can ever acquire a value of any attribute in any tuple that is not the appropriate type.

Type Constraints...

e.g. Use check clause to ensure that an hourly-wage domain allows only values greater than a specified value.

create domain *hourly-wage* numeric(5,2)

constraint *value-test* check(value >= 4.00)

- The domain *hourly-wage* is declared to be a decimal number with 5 digits, 2 of which are after the decimal point.
- The domain has a constraint that ensures that the hourly-wage is greater than 4.00.
- The clause *constraint value-test* is optional; useful to indicate which constraint an update violated.

Type Constraints...

/*先定义一个名为SexDomain的域*/

```
CREATE DOMAIN SexDomain CHAR(1)
```

```
    CHECK (VALUE IN ('F' , 'M'));
```

/*再用ALTER TABLE语句修改Customer表*/

```
ALTER TABLE Customer
```

```
ADD Contact_Psex SexDomain;
```


3.Attribute constraint

- An attribute constraint is basically just a declaration to the effect that **a specified attribute is of a specified type.**

e.g.

```
create table account  
  (branch-name      char(15),  
   account-number char(10) not null,  
   balance integer,  
   .....)
```

Attribute constraint...

- Attribute constraints are part of the definition of the attribute.
- Attribute constraints are always **checked immediately**.
- Any attempt to introduce an attribute value into the database that is not a value of the relevant type will simply be rejected.

Attribute constraint...

CREATE TABLE Order

(Ono CHAR(4),

Cno CHAR(4) NOT NULL,

Odate DATE,

Freight INT CHECK (Freight BETWEEN 0 AND 1000),

...);

4. Relvar Constraints

- A relvar constraint is a constraint on an individual relvar.
- e.g. suppliers in London must have status 20
CONSTRAINT SC5
IS_EMPTY (CITY='LONDON' AND STATUS!=20);
- Relvar constraints are always **checked immediately**.
- Any statement that attempts to assign a value to a given relvar that violates any relvar constraint for that relvar will effectively just be rejected.

Relvar Constraints-SQL

CREATE TABLE Ewage

(Eno CHAR(1), /*职工号*/

Salary INT, /*基本工资*/

Allowance INT, /*津贴*/

Bonus INT, /*奖金*/

PRIMARY KEY(Eno),

CHECK (Bonus>0 AND Bonus<3*Salary)

/*奖金不得高于基本工资的 3 倍*/

);

5.Database Constraints

- A database constraint is a constraint that interrelates two or more distinct relvars.
- e.g. no supplier with status less than 20 can supply any part in a quantity greater than 500.

CONSTRAINT DBC1

IS_EMPTY ((S JOIN SP)

WHERE STATUS<20 AND QTY>500);

- In general database constraint *checking must be deferred to end-of-transaction.*
- If a database constraint is violated at COMMIT time, the transaction is rolled back.

6. The Golden Rule(黄金法则)

- Any given relation has an associated predicate(谓词). A relvar too has a predicate.
- Which serves as the criterion for acceptability of updates on the relvar
- Ideally, the DBMS would know and understand the predicate for every relvar, but this goal is unachievable. The system does not know and understand predicate 100 percent. But, it does know a good approximation to that predicate. It knows the integrity constraints that apply to suppliers.
- We define the **relvar predicate** for the supplier relvar to be the logical AND of all relvar constraints that apply to that relvar.

The Golden Rule...

e.g.

(IS_EMPTY(S WHERE STATUS<1 OR
STATUS>100) **AND**

(IS_EMPTY(S WHERE CITY='London' AND
STATUS ≠20)) **AND**

(COUNT(S)=COUNT(S(S#)))

informal (external) predicate $\Leftrightarrow$ formal (internal)

The Golden Rule...

The Golden rule:

- *No update operation must ever be allowed to leave any relvar in a state that violate its own predicate.*
- *No update transaction must ever be allowed to leave database in a state that violate its own predicate.*

The golden rule applies to all relvars.

7. State vs. Transition Constraints (静态和动态约束)

- All of the constraints discussed in this chapter so far have been state constraints.
- **State constraints:**
Concerned with correct states of the database.
- **Transition constraints :**
on legal transitions from one correct state to another.

State vs. Transition Constraints...

- The following transitions are all valid:
 - Never married to married
 - Married to widowed
 - Married to divorced
 - Widowed to married
- The following transitions are not valid:
 - Never married to widowed
 - Never married to divorced
 - Widowed to divorced
 - Divorced to widowed

State vs. Transition Constraints...

- e.g. No supplier's status must ever decrease.

```
CONSTRAINT TRC1 IS EMPTY
( ( ( S' { S#, STATUS } RENAME STATUS AS STATUS' )
  JOIN S { S#, STATUS } )
  WHERE STATUS' > STATUS ) ;
```

- S' – old value
- *TRC1* is a relvar transition constraint.
- *TRC1* applies to just a single relvar (suppliers)
- *TRC1* is **checked immediately**.

8. Keys

- Candidate Keys and superkey
- Primary Keys and Alternate Keys
- Foreign Keys

Candidate Keys

- Let K be a set of attributes of relvar R . Then K is a **candidate key** for R if and only if it possesses both of the following properties:
 - **Uniqueness**: No legal value of R ever contains two distinct tuples with the same value for K .
 - **Irreducibility(不可约的)**: No proper subset of K has the uniqueness property.

```
VAR MARRIAGE BASE RELATION { HUSBAND      NAME,  
                             WIFE         NAME,  
                             DATE /* of marriage */ DATE }  
/* assume no polyandry, no polygyny, and no husband and */  
/* wife marry each other more than once ... */  
    KEY { HUSBAND, DATE }  
    KEY { DATE, WIFE }  
    KEY { WIFE, HUSBAND } ;
```

Candidate Keys...

```
VAR S BASE RELATION
  { S#      S#,
    SNAME   NAME,
    STATUS  INTEGER,
    CITY    CHAR }
  KEY { S# } ;
```

- Every relvar does have at least one candidate key
- A candidate key definition is really just shorthand for a certain relvar constraint. Useful for provide the basic **tuple – level addressing mechanism** in the relational model
- A superset of a candidate key is a **superkey**. For example, the set of attributes {S#,CITY} is a superkey for relvar S.

Primary Keys and Alternate Keys

- It is possible for a given relvar to have more than one candidate key. One of those keys be chosen as the **primary key**, and the others are then called **alternate keys**.

e.g.

create table *customer*

```
(customer-name    char(20) not null,  
customer-street   char(30),  
                  customer-city    char(30),  
primary key (customer-name));
```

create table *branch*

```
(branch-name char(15) not null,  
branch-city char(30),  
assets integer,  
primary key (branch-name));
```


Foreign Keys

- Let R_2 be a relvar. Then a **foreign key** in R_2 is a set of attributes of R_2 , say FK , such that:
 - There exists a relvar R_1 (R_1 and R_2 not necessarily distinct) with a candidate key CK , and
 - For all time, each value of FK in the current value of R_2 is identical to the value of CK in some tuple in the current value of R_1 .
- Every value of a given foreign key to appear as a value of the matching candidate key. The converse is not a requirement.
- A foreign key is simple or composite according as the candidate key it matches
- Terms: reference; referenced tuple; referential integrity problem; referential constraint; referencing relvar, referenced relvar

Referential Integrity in SQL - Example

create table *account*

 (*branch-name* char(15),
account-number char(10) **not null**,
 balance integer,
primary key (*account-number*),
foreign key (*branch-name*) **references** *branch*);

create table *depositor*

(*customer-name* char(20) **not null**,
account-number char(10) **not null**,
primary key (*customer-name*, *account-number*),
foreign key (*account-number*) **references** *account*,
foreign key (*customer-name*) **references** *customer*);

9. Referential Actions

- DELETE/UPDATE rule for a particular foreign key:
 - CASCADE
 - RESTRICT (NO ACTION)
 - SET NULL
 - SET DEFAULT

Delete Rules - ON DELETE CASCADE

DEP	DEPNAME	MANAGER	DIVISION
BLU	PLANNING	000020	EASTERN
GRE	OPERATIONS	000080	WESTERN
RED	MARKETING	000010	EASTERN

EMPNO	LASTNAME	FIRSTNAME	DEP	GOVT_ID	SALARY
000010	HAAS	CHRISTINE	RED	888-88-2794	52750.00
000020	THOMPSON	MICHAEL	BLU	888-89-4261	31000.00
000030	KWAN	SALLY	GRE	888-88-9456	33000.00

DEP	DEPNAME	MANAGER	DIVISION
GRE	OPERATIONS	000080	WESTERN
RED	MARKETING	000010	EASTERN

EMPNO	LASTNAME	FIRSTNAME	DEP	GOVT_ID	SALARY
000010	HAAS	CHRISTINE	RED	888-88-2794	52750.00
000030	KWAN	SALLY	GRE	888-88-9456	33000.00

Delete Rules - ON DELETE RESTRICT

DEP	DEPNAME	MANAGER	DIVISION
BLU	PLANNING	000020	EASTERN
GRE	OPERATIONS	000080	WESTERN
RED	MARKETING	000010	EASTERN

EMPNO	LASTNAME	FIRSTNAME	DEP	GOVT_ID	SALARY
000010	HAAS	CHRISTINE	RED	888-88-2794	52750.00
000020	THOMPSON	MICHAEL	BLU	888-89-4261	31000.00
000030	KWAN	SALLY	GRE	888-88-9456	33000.00

DEP	DEPNAME	MANAGER	DIVISION
BLU	PLANNING	000020	EASTERN
GRE	OPERATIONS	000080	WESTERN
RED	MARKETING	000010	EASTERN

EMPNO	LASTNAME	FIRSTNAME	DEP	SOC_SEC	SALARY
000010	HAAS	CHRISTINE	RED	888-88-2794	52750.00
000020	THOMPSON	MICHAEL	BLU	888-89-4261	31000.00
000030	KWAN	SALLY	GRE	888-88-9456	33000.00

Referential Actions

```
CREATE TABLE SP
(S#   S# NOT NULL,
P#   P# NOT NULL,
QTY   QTY NOT NULL,
PRIMARY KEY(S#,P#),
FOREIGN KEY (S#) REFERENCE S
            ON DELETE CASCADE
            ON UPDATE CASCADE
FOREIGN KEY (P#) REFERENCE P
            ON DELETE CASCADE
            ON UPDATE CASCADE
CHECK(QTY>0 AND QTY<5001));
```

Referential Actions...

create table account

(...

foreign key(branch-name) references branch

on delete cascade

on update cascade.

...)

- Due to the on delete cascade clauses, if a delete of a tuple in branch results in referential-integrity constraint violation, the delete “cascades” to the account relation, deleting the tuple that refers to the branch that was deleted.
- Cascading updates are similar.

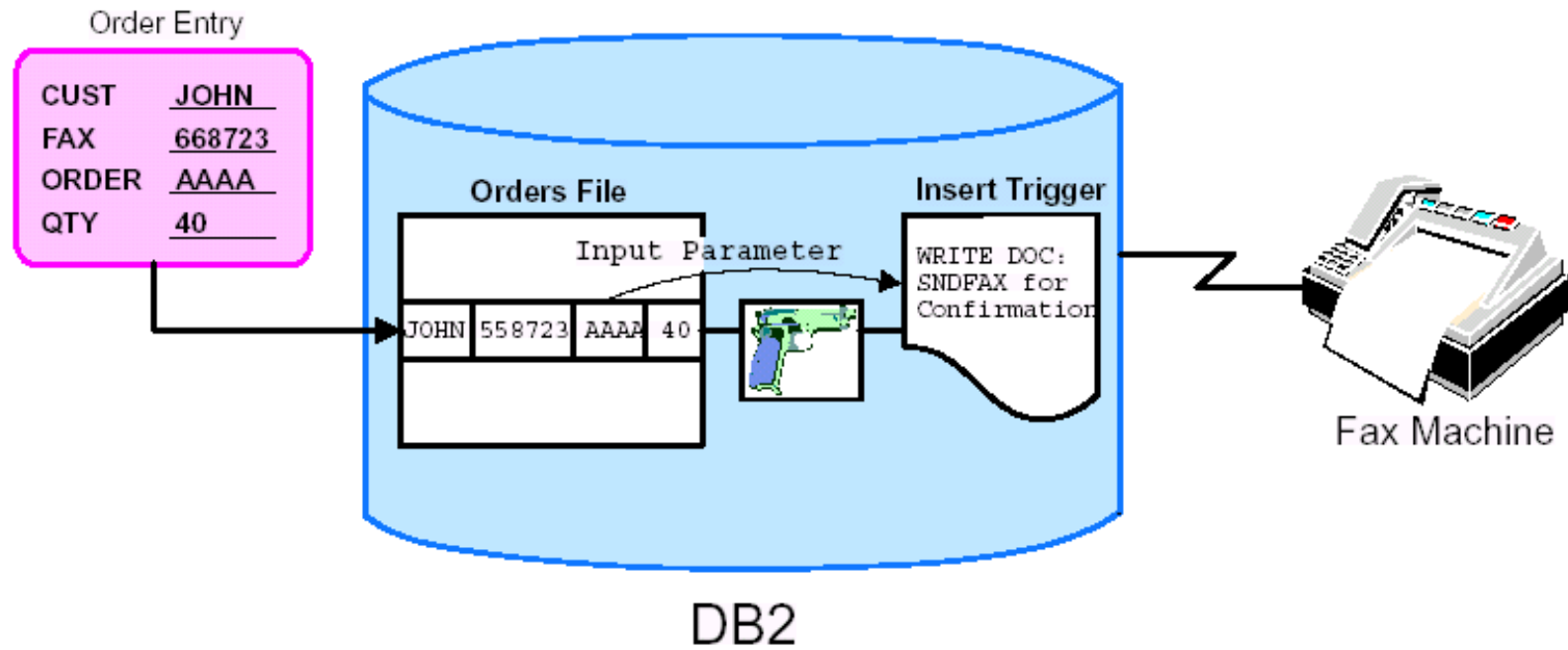
Referential Actions...

- If there is a chain of foreign-key dependencies across multiple relations, with on delete cascade specified for each dependency, a deletion or update at one end of the chain can propagate across the entire chain.
- If a cascading update or delete causes a constraint violation that cannot be handled by a further cascading operation, the system aborts the transaction. As a result, all the changes caused by the transaction and its cascading actions are undone.

10. Triggers

- A trigger (触发器) is a statement that is executed automatically by the system as a side effect of a modification to the database.
- To design a trigger mechanism, we must:
 - Specify the conditions under which the trigger is to be executed.
 - Specify the actions to be taken when the trigger executes.
- The SQL standard does not include trigger, but many implementations support triggers.

Triggers: A Sophisticated Example



- When a new order is inserted, a trigger starts
- Triggers get information about order and customer
- A confirmation fax is automatically sent

Triggers: A Simple Example

```
CREATE TRIGGER NEW_HIRED  
  AFTER INSERT ON EMP  
  FOR EACH ROW  
  MODE DB2SQL  
  UPDATE COMPANY_STATS SET NBEMP = NBEMP + 1
```

```
CREATE TRIGGER FORMER_EMP  
  AFTER DELETE ON EMP  
  FOR EACH ROW  
  MODE DB2SQL  
  UPDATE COMPANY_STATS SET NBEMP = NBEMP - 1
```

Triggers-Example

- Suppose that instead of allowing negative account balances, the bank deals with overdrafts by
 - setting the account balance to zero
 - creating a loan in the amount of the overdraft
 - giving this loan a loan number identical to the account number of the overdrawn account
- The condition for executing the trigger is an update to the account relation that results in a negative balance value.

Triggers-Example...

define trigger *overdraft* on update of *account T*

(if *new T.balance* < 0

then (insert into *loan* values

(*T.branch-name*, *T.account-number*, *-newT.balance*)

insert into *borrower*

(select *customer-name*, *account-number*

from *depositor*

where *T.account-number* = *depositor.account-number*)

update *account.S*

set *S.balance* = 0

where *S.account-number* = *T.Account-number*))

- The keyword *new* used before *T.balance* indicates that the value of *T.balance* after the update should be used; if it is omitted, the value before the update is used.

Banking Example

branch		
branch-name	branch-city	assets
Downtown	Brooklyn	9000000
Redwood	Palo Alto	2100000
Perryridge	Horseneck	1700000
Mianus	Horseneck	400000
Round Hill	Horseneck	8000000
Pownal	Bennington	300000
North Town	Rye	3700000
Brighton	Brooklyn	7100000

account		
branch-name	account-number	balance
Downtown	A-101	500
Mianus	A-215	700
Perryridge	A-102	400
Round Hill	A-305	350
Brighton	A-201	900
Redwood	A-222	700
Brighton	A-217	750

borrow	
customer-name	loan-number
Jones	L-17
Smith	L-23
Hayes	L-15
Johnson	L-14
Curry	L-93
Smith	L-11
Williams	L-17
Adams	L-16

customer		
customer-name	customer-street	customer-city
Jones	Main	Harrison
Smith	North	Rye
Hayes	Main	Harrison
Curry	North	Rye
Lindsay	Park	Pittsfield
Turner	Putnam	Stamford
Williams	Nassau	Princeton
Adams	Spring	Pittsfield
Johnson	Alma	Palo Alto
Glenn	Sand Hill	Woodside
Brooks	Senator	Brooklyn
Green	Walnut	Stamford

loan		
branch-name	loan-number	amount
Downtown	L-17	1000
Redwood	L-23	2000
Perryridge	L-15	1500
Downtown	L-14	1500
Mianus	L-93	500
Round Hill	L-11	900
Perryridge	L-16	1300

depositor	
customer-name	account-number
Johnson	A-101
Smith	A-215
Hayes	A-102
Turner	A-305
Johnson	A-201
Jones	A-217
Lindsay	A-222

11. Assertions

- An assertion (断言) is a predicate expressing a condition that we wish the database always to satisfy.
- An assertion in SQL takes the form

```
create    assertion    <assertion-name>    check  
<predicate>
```
- When an assertion is made, the system tests it for validity. This testing may introduce a significant amount of overhead; hence assertions should be used with great care.

Assertions

Create assertion IC11 check

```
((select min(S.status) from S ) >4);
```

Create assertion IC18 check

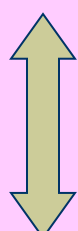
```
(not exist ( select * from P  
Where not (P.weight>0)));
```


Assertions...

```
ALTER TABLE Customer  
ADD Discount_Flag CHAR(1) CHECK (VALUE IN('Y',  
'N')));  
CREATE ASSERTION DiscountCustomer CHECK  
(NOT EXISTS  
  (SELECT *  
    FROM Customer Cx  
    WHERE Cx.Discount_Flag="Y" AND  
      (20>=  
        (SELECT COUNT(*)  
          FROM Order Ox  
          WHERE Ox.Cno=Cx.Cno ) ));
```

Assertions...

```
CREATE ASSERTION Cno_GZ CHECK
(NOT EXISTS
 (SELECT *
  FROM Customer,Order_GZ
  WHERE Customer.Cno=Order_GZ.Cno AND
        Customer.City<>"广州"
 )
);
```



```
CREATE TABLE Order_GZ(...
CONSTRAINT CS1 CHECK (Cno IN (SELECT Cno FROM
Customer WHERE City="广州" ) ) ...);
```

Assertions...

- Dropping an assertion in SQL takes the form
DROP assertion <assertion-name>

Does not offer a restrict vs cascade option.

In SQL, constraints can be defined to be Deferrable / not deferrable

not deferrable -> always check immediately

Deferrable-> can be switched on and off by

Set constraints IC46,IC95 deferred

Set constraints IC46,IC95 immediate

COMMIT forces a “set * immediate” for all deferrable constraints

SQL facilities

SQL classifies constraints into three broad categories, as follows:

- Domain constraints
- Base table constraints
- General constraints (assertion)

SQL has no support at all for transition constraints, nor does it support triggered procedure

Practical

- 指出上次作业描述的关系模型中各关系模式的主码、候选码、外部码
- 分析在上次作业描述的关系模型中，存在哪些完整性要求；
- 分析在你的关系模型中，需要定义那些用户视图？并说明其作用；
- 对所定义的视图，更新时有无限制，并举例说明。
- 使用SQL定义该关系模型（包括用户视图）；
- 在DB定义中，主码、外部码、引用完整性、用户完整性等是如何体现的；
- 在你的数据库中，有哪些常见的查询操作？分别用关系代数和SQL语言实现之；